A interface **`Callable`** (do pacote `java.util.concurrent`, introduzida no Java 5) representa uma **tarefa que pode ser executada por outra *thread***, de forma semelhante à tradicional `Runnable`, mas com duas grandes vantagens: **retorna um valor** e **pode lançar exceções verificadas** (*checked exceptions*). [1, 2, 3, 4, 5]

Principais Características

- **Assinatura do Método:** Possui apenas um método abstrato (sendo, portanto, uma *interface funcional*):
  java
  ```
  @FunctionalInterface
  public interface Callable<V> {
      V call() throws Exception;
  }
  ```
  Use o código com cuidado.
- **Retorno de Valor (`V`):** Utiliza Generics para permitir que a tarefa calcule e devolva um resultado ao término do processamento.
- **Lançamento de Exceções:** O método `call()` declara `throws Exception`, permitindo que você lide diretamente com erros (como `IOException` ou `SQLException`) dentro da tarefa assíncrona.
- **Uso com `ExecutorService` e `Future`:** Diferente da `Runnable` (que pode ser passada direto para um objeto `Thread`), a `Callable` é enviada a um `ExecutorService` através do método `.submit()`. Esse método retorna um objeto **`Future`**, que serve como uma promessa para recuperar o resultado da tarefa quando ela terminar. [1, 2, 3, 4, 5, 6, 7, 8]

`Runnable` vs. `Callable`

Característica	**`Runnable`**	**`Callable`**
Pacote	`java.lang`	`java.util.concurrent`
Método principal	`run()`	`call()`
Retorna valor?	Não (`void`)	Sim (tipo genérico `V`)
Lança *checked exceptions*?	Não	Sim (`throws Exceptio`

Execução direta em `Thread`?	Sim (`new Thread(runnable).start()`)	Não (exige `ExecutorService.subm`

Exemplo Prático

java

```java
import java.util.concurrent.*;

public class ExemploCallable {
    public static void main(String[] args) {
        // Cria um pool com uma única thread
        ExecutorService executor = Executors.newSingleThreadExecutor();

        // Submete a Callable
        Future<Integer> futuroResultado = executor.submit(new
Callable<Integer>() {
            @Override
            public Integer call() throws Exception {
                Thread.sleep(1000); // Simula uma operação demorada
                return 42; // Retorna um resultado
            }
        });

        try {
```

Use o código com cuidado.